

1. SGLang 的 RadixAttention / RadixCache 解决什么问题？

自回归推理里每个 token 的 Attention 都依赖前缀的 K/V。传统服务把 KV 绑在单次请求生命周期上，请求结束就释放，系统提示、多轮对话、分叉生成等共享前缀会被反复 Prefill。RadixCache 用基数树把 token 序列映射到 paged KV 槽位；RadixAttention 是在这棵树上做前缀匹配与注意力计算的机制。match_prefix 找到最长已缓存前缀后，本轮只需对未命中后缀做 Prefill，从而降低重复计算、提高吞吐并缩短共享前缀场景的 TTFT。

2. page-sized KV cache 与 prefix reuse 分别对应 pipeline 的哪一部分？

page-sized KV cache 对应内存分配层：token_to_kv_pool_allocator / PagedTokenToKVPoolAllocator，在 Prefill Adder 为未缓存 token 分配 page 对齐槽位，Decode 阶段按 page 读写 K/V。它解决显存碎片与动态长度，不负责跨请求索引。prefix reuse 对应调度与缓存索引层：Scheduler.get_new_batch_prefill -> match_prefix_for_req -> RadixCache.match_prefix，用命中长度缩短 extend_input_len；请求结束后 cache_finished_req / cache_unfinished_req 把新 K/V 插回基数树，供后续请求复用。

3. vLLM PagedAttention 与 SGLang RadixAttention / paged KV 的联系与差异

联系：二者都把 KV 从连续张量改成按 block/page 管理，用块表做逻辑到物理映射，从而在连续批处理中高效共享显存。差异：PagedAttention (SOSP 2023) 首先解决单机批内显存碎片与内部浪费，前缀缓存是后来在定长 block 哈希表上叠加的；RadixAttention (NeurIPS 2024) 把跨请求前缀复用做成一等公民，用可变长边的基数树表达包含关系，命中可到 token 级 (page_size=1) 并在部分命中时 split node。vLLM 偏块级哈希精确匹配，SGLang 偏树形最长前缀匹配加引用计数淘汰。